

Vectorised partial gamma for DIF

Validation against `iarm::partgam_DIF()` before any package change

Magnus Johansson

2026-10-03

Table of contents

Purpose	1
Validation matrix	1
Timing	4
Findings	5

Purpose

`RMlocdepGamma()` and `RMlocdepGammaCutoff()` already use a vectorised partial gamma (`.partgam_ld_gamma()` and `.partgam_one()` in `R/ld_partgam.R`), validated exactly against `iarm::partgam_LD()`. `RMdifGammaCutoff()` still calls `iarm::partgam_DIF()` once per bootstrap iteration. The DIF case differs from the LD case in four ways the LD validation does not cover (`dev/dif-impact-design.md`, Part A, question 4):

1. The table is rectangular, item categories by groups, not item by item.
2. The conditioning variable is the full score, the item included.
3. The grouping variable is a factor, and `iarm::partgam()` orders it by level, so the sign depends on level order and three or more groups are treated as ordinal.
4. Missing data: `iarm::partgam_DIF()` drops incomplete rows (items or grouping variable).

The prototype is `partgam_dif_fast()` in `dev/dif_partgam_fast.R`. This note checks it against `iarm::partgam_DIF()` over a validation matrix, against a brute-force pair count where `iarm` errors, and times both.

```
source("dif_partgam_fast.R")
source("dif_shift_cml.R") # for sim_pcm() and sim_dif()
set.seed(20261003)

thr_items <- function(locs, steps) {
  out <- lapply(locs, function(l) l + steps)
  names(out) <- paste0("I", seq_along(locs))
  out
}
thr9 <- thr_items(seq(-1.5, 1.5, length.out = 9), c(-0.8, 0, 0.8))
```

Validation matrix

Each case gives a data matrix and a grouping variable. For every case the table reports the largest absolute difference between the fast and `iarm` gammas over items, and the same against the brute-force count. `iarm` returns its results through a character data.frame, so agreement to about 15 significant digits is the best attainable against it. The brute-force count is exact.

```

mk <- function(n1, n2, thr = thr9, dif = c(I3 = 0.6), mu2 = 0.3) {
  sim_dif(n1, n2, thr, dif = dif, mu2 = mu2)
}
cases <- list()

# Baseline, polytomous, with DIF so that gammas are not all near zero
for (n in c(200, 600, 1000)) {
  s <- mk(n / 2, n / 2)
  cases[[paste0("poly4_n", n)]] <- list(X = s$X, g = s$g)
}

# Level order reversed: same data, factor levels swapped
s <- mk(300, 300)
cases$levels_reversed <- list(X = s$X, g = factor(s$g, levels = c("g2", "g1")))

# Character and numeric codings of the same grouping
cases$character <- list(X = s$X, g = as.character(s$g))
cases$numeric_01 <- list(X = s$X, g = as.integer(s$g) - 1L)
cases$numeric_12 <- list(X = s$X, g = as.integer(s$g))

# Factor with an unused level
cases$unused_level <- list(X = s$X, g = factor(s$g, levels = c("g1", "g2", "g3")))

# Unequal groups, 90/10
s <- mk(540, 60)
cases$unequal_90_10 <- list(X = s$X, g = s$g)

# A group absent from some strata: large group difference
s <- mk(300, 300, mu2 = 2.5)
cases$group_absent_in_strata <- list(X = s$X, g = s$g)

# Dichotomous, 20 items
s <- mk(200, 200, thr = thr_items(seq(-2, 2, length.out = 20), 0), dif = c(I5 = 0.6))
cases$dich_k20 <- list(X = s$X, g = s$g)

# Mixed category counts
thr_mix <- c(thr_items(c(-1, 0, 1), 0),
             setNames(thr_items(c(-0.5, 0.5), c(-0.6, 0.6)), c("I4", "I5")),
             setNames(thr_items(c(-0.8, 0.8), c(-1, 0, 1)), c("I6", "I7")))
s <- mk(250, 250, thr = thr_mix, dif = c(I6 = 0.6))
cases$mixed_categories <- list(X = s$X, g = s$g)

# An unobserved middle category in one item
s <- mk(250, 250)
s$X[s$X[, "I2"] == 1L, "I2"] <- 2L
cases$unobserved_middle <- list(X = s$X, g = s$g)

# Sparse strata
for (n in c(50, 30)) {
  s <- mk(n / 2, n / 2)
  cases[[paste0("sparse_n", n)]] <- list(X = s$X, g = s$g)
}

# Missing data: 10% of rows with an item NA, and NA in the grouping variable

```

```

s <- mk(300, 300)
Xna <- s$X
rows <- sample(nrow(Xna), 60)
Xna[cbind(rows, sample(ncol(Xna), 60, replace = TRUE))] <- NA
gna <- s$g
gna[sample(nrow(Xna), 30)] <- NA
cases$na_items <- list(X = Xna, g = s$g)
cases$na_group <- list(X = s$X, g = gna)
cases$na_both <- list(X = Xna, g = gna)

# Three groups, treated as ordinal in level order
s <- mk(300, 300)
g3 <- factor(sample(c("a", "b", "c"), nrow(s$X), replace = TRUE))
cases$three_groups <- list(X = s$X, g = g3)

# A constant item: iarm is expected to error
s <- mk(200, 200)
Xc <- s$X
Xc[, "I9"] <- 0L
cases$constant_item <- list(X = Xc, g = s$g)

```

```

val <- do.call(rbind, lapply(names(cases), function(nm) {
  cs <- cases[[nm]]
  fast <- partgam_dif_fast(cs$X, cs$g)$gamma
  brute <- partgam_dif_brute(cs$X, cs$g)
  ia <- partgam_dif_iarm(cs$X, cs$g)
  data.frame(
    case = nm,
    n_used = sum(stats::complete.cases(cs$X) & !is.na(cs$g)),
    max_diff_iarm = if (is.null(ia)) NA_real_ else max(abs(fast - ia), na.rm = TRUE),
    max_diff_brute = max(abs(fast - brute), na.rm = TRUE),
    na_match = identical(is.na(fast), is.na(brute)),
    iarm_error = is.null(ia),
    max_abs_gamma = max(abs(fast), na.rm = TRUE)
  )
}))
val

```

case	n_used	max_diff_iarm	max_diff_brute	na_match	iarman_error	max_abs_gamma
poly4_n200	200	0	0	TRUE	FALSE	0.4812287
poly4_n600	600	0	0	TRUE	FALSE	0.3333333
poly4_n1000	1000	0	0	TRUE	FALSE	0.4080647
levels_reversed	600	0	0	TRUE	FALSE	0.3419405
character	600	0	0	TRUE	FALSE	0.3419405
numeric_01	600	0	0	TRUE	FALSE	0.3419405
numeric_12	600	0	0	TRUE	FALSE	0.3419405
unused_level	600	0	0	TRUE	FALSE	0.3419405
unequal_90_10	600	0	0	TRUE	FALSE	0.4169096
group_absent_in_strata	600	0	0	TRUE	FALSE	0.2163743

case	n_used	max_diff_iarm	max_diff_brute	na_matchiarm_error	max_abs_gamma	
dich_k20	400	0	0	TRUE	FALSE	0.2838284
mixed_categories	500	0	0	TRUE	FALSE	0.3095965
unobserved_middle	500	0	0	TRUE	FALSE	0.3343849
sparse_n50	50	0	0	TRUE	FALSE	0.8095238
sparse_n30	30	0	0	TRUE	FALSE	1.0000000
na_items	540	0	0	TRUE	FALSE	0.4350678
na_group	570	0	0	TRUE	FALSE	0.4293717
na_both	512	0	0	TRUE	FALSE	0.4295400
three_groups	600	0	0	TRUE	FALSE	0.1008206
constant_item	400	NA	0	TRUE	TRUE	0.3833333

`max_abs_gamma` is there to show that the comparisons are not trivially between zeros. For `constant_item`, `iarm` is expected to error for the whole dataset (`iarm_error = TRUE`), the fast version should return `NA` for the constant item only, and the brute-force count closes the case.

```
# The reversed-level case must be the exact negative of the baseline order.
s_rev <- cases$levels_reversed
c(
  sign_flip_exact = isTRUE(all.equal(
    partgam_dif_fast(s_rev$X, factor(s_rev$g, levels = c("g1", "g2")))$gamma,
    -partgam_dif_fast(s_rev$X, s_rev$g)$gamma, tolerance = 0
  ))
)
```

```
sign_flip_exact
      TRUE
```

Timing

One call per method on the `n = 600` baseline, repeated. Each pass evaluates the call afresh (a `force()` inside a timing loop evaluates once and caches, which divided an earlier LD benchmark by the number of repetitions).

```
time_it <- function(expr, times = 20L) {
  e <- substitute(expr)
  pf <- parent.frame()
  t0 <- proc.time()[["elapsed"]]
  for (i in seq_len(times)) eval(e, pf)
  1000 * (proc.time()[["elapsed"]] - t0) / times
}
cs <- cases$poly4_n600
data.frame(
  method = c("iarm::partgam_DIF()", "partgam_dif_fast()"),
  ms_per_call = c(
    time_it(partgam_dif_iarm(cs$X, cs$g)),
    time_it(partgam_dif_fast(cs$X, cs$g))
  )
)
```

method	ms_per_call
iarm::partgam_DIF()	27.55
partgam_dif_fast()	0.55

Findings

Run 2026-10-03.

1. **Exact agreement.** The fast version equals `iarm::partgam_DIF()` (difference 0) in all 19 cases iarm can compute, and the brute-force pair count in all 20. This covers the rectangular table, the full-score conditioning, factor, character and numeric coding of the grouping variable, reversed and unused levels, 90/10 groups, a group absent from some strata, sparse samples down to $n = 30$, missing values in items and in the grouping variable, and three groups treated as ordinal.
2. **Sign follows level order.** Reversing the levels negates every gamma exactly, as in iarm.
3. **A constant item no longer sinks the whole analysis.** iarm errors for the whole dataset. The fast version returns `NA` for the constant item only, confirmed by the brute-force count.
4. **About 65 times faster.** 0.5 ms against 33 ms per call at $n = 600$ with nine items.

The prototype can replace `iarm::partgam_DIF()` inside `RMdifGammaCutoff()` and for the observed statistic in `RMdifGamma()`, as the LD functions already do, so that the null and the observed value come from the same code. Package change waits for the decision on Part A of the design note.